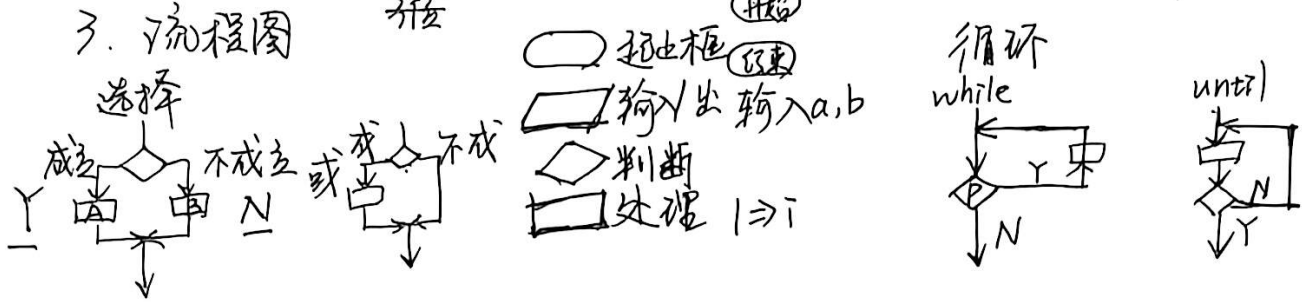


1. 运算符优先级: 单目 > 双目算术 > 双目 == / != > 逻辑 > 赋值
 2. #include <stdio.h> scanf("%f", &f) 12.34e3 > 12.34 x 10^3 (e 后为整数)
 保留三位小数 %3f \t 水平制表符 #define PRICE 40 float a = 3.14159f



4. C 语句: 控制语句、函数调用语句、表达式语句、空、复合。

5. 输入输出 printf scanf
 %5d 占5列 12 -345 %5.3f 保留3位小数 %c %s
 向右对齐 %-m.nf %e 指数

putchar(c) c='B' 单引! 输出一个 字符也是整型
 getch() 输入一个 a=getchar();
 puts(字符串数组); gets(str) 输入串(一个)

6. 三目: A? B: C; max=(a>b)? a: b
 真假 a>b a≤b

7. switch(N)
 {
 case 1: ; break;
 case 2: ; break;
 ...
 default:
 }
 共用P case1:
 case2:
 case3: p; break;

8. break, continue

9. 数组中 a[n] n 不能为变量 默认: 0, '\0', NULL

二维 行 列
 char C[] > printf("%s\n", C)



10. 常用字符串处理函数 `#include <string.h>`

`strcat` (字符串数组1, 2) 连接 将2接到1中

`strcpy` (1, 2) 复制 2复制到1中

~~`strncpy`~~ `strncpy` (1, 2, n) 将2中前n个字符复制到1

`strcmp` (1, 2) 比较 $\begin{cases} = 0 \\ > 0 \\ < 0 \end{cases}$

`strlen` (数组) 长度

11. ① void 无返回 `return` ② 形参临时 ③ 先声明后使用 (全局/局部)

④ 函数递归调用

⑤ 形参数组可以不指定大小 `array[]`

`int fac(int n)`

⑥ 不同函数可以使用相同变量名

{

`fac(...)` 无返回值

}

① 12 指针是一个地址 `*p` 代表 `p` 指向的对象

定义 `int a, *p; int a, b ...`
`p = &a; int *p1 = a, *p2 = b;`

② 一维 `p = a / a[0] / a[...]` `p+1` 表示下一个 $\frac{*(p+1)}{*(a+1)} = a[i]$

`p++` 自右向左
`*p++` `p` 的值+1

③ 二维 `*(a+1)+2` 1行2列 $\frac{*(*(a+1)+2)}{1}$ 值

④ 指向行 `int a[4] = ... int (*p)[4] p = &a`

⑤ 指向函数 `int max(); p = max;`
`int (*p)();`

⑥ 返回指针的函数

类型 变量名 (参);

`C = (*p)();` 调用
`float *search()`
{
 float *p;
 return(p);
}

⑦ 指针数组 (放地址)

`int *p[4];`

⑧ `char **p`

13. 动态内存分配 `#include <stdlib.h>`

`malloc(100)` 开辟100字节

`calloc(50, 4)` 50x4

`realloc(p, 50)` `p` 改为50

`free(p)` 释放 `p`

`malloc-free`

最好成对出现

避免内存泄漏

14. 定义结构体

① 定义 `struct ...`

`struct ...`
{
 a, b;
};

`a = { ... };`

`struct ...`

{
 leader[3] = { ... };
}

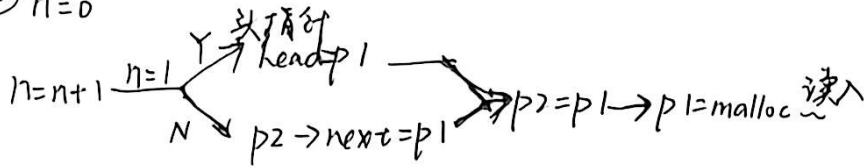
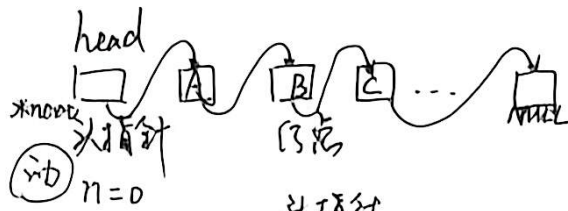
(结构体数组)

② 指针 `struct ... *p;`

③ $\frac{p \rightarrow \text{num}}{\text{stu.num}}$ / $(*p).num$



13. 链表



建立

#define LEN sizeof (struct Student)

struct Student {

int n;

struct Student *next; 返回指向head的指针

```

{
    n=0;
    p1=p2=(struct Student*) malloc (LEN);
    scanf("...", &p1->, &p1-> );
    head=NULL;
    while (p1->num!=0)
    {
        n=n+1;
        if (n==1) head=p1;
        else p2->next=p1;
        p2=p1;
        p1=(struct Student*) malloc (LEN);
        scanf("...", p1 );
    }
    p2->next=NULL;
    return (head);
}

```

main: int *p struct Student *pt=creat()

输出

void print (struct Student *head)

```

{
    struct S *p;
    p=head;
    if (head!=NULL) 不是空表
    do
    {
        printf("%ld %5.1f\n", p->num, p->score);
        p=p->next;
    } while (p!=NULL);
}

```



anz: $a \geq 'a' \ \&\& \ a \leq 'z'$

qsort 排序: $qsort(*p, n, size, *compare)$
指向排数量 每个元素 比较函数
数组 大小 指针: $1 < 2 < 0$
 $1 = 2 = 0$
 $1 > 2 > 0$

链表就地逆置

```
*pre = NULL *now = head *next = NULL
while (*now != NULL)
{
    next = now->next; 保存下个
    now->next = pre; 反转当前
    pre = now;
    now = next; > 向后移动
}
return pre;
```

$\boxed{pre} \ \boxed{now} \rightarrow NULL$

选择排序

小大 选最小与 $a[0]$ 交换, 第二小与 $a[1]$...

```
for (int i = 0; i < n - 1; i++)
{
    k = i;
    for (j = i + 1; j < n; j++)
    {
        if (a[j] < a[k])
            k = j; 选最小
    }
    t = a[k];
    a[k] = a[i]; 换
    a[i] = t;
}
```

